

BHTTP/1: Binary HyperText Protocol, version 1

Siddhant Prasad · Network Architecture course project · spec v1.0

The key words MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. All integers are unsigned and big-endian (network byte order). Not related to RFC 9292 "Binary HTTP".

1. Model

BHTTP carries HTTP semantics (methods, paths, status codes, header fields, bodies) over one TCP connection as **binary frames**. A client sends a REQUEST. The server answers with one RESPONSE followed by zero or more DATA frames. The connection is persistent, so any number of requests may follow on it. Each request gets a stream id. In v1 the server handles requests strictly in order, one at a time.

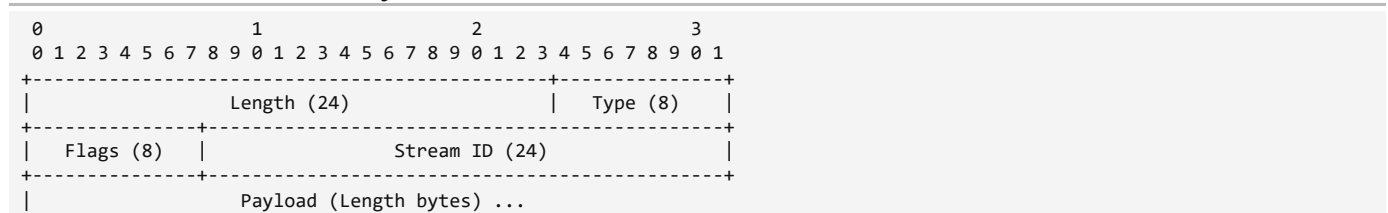
Default port: **9000**. URL form: `bhttp://host[:port]/path`.

2. Connection preface

Each side starts with the same 8 bytes: `89 42 48 54 50 0D 0A 01`. That is `0x89`, then "BHTTP", then CR LF, then the **version** byte (`01`). The client sends its preface first. It MAY send its first REQUEST right away without waiting. The server checks the client's preface, then sends its own.

- If the magic is wrong, the server sends its preface and a GOAWAY PROTOCOL_ERROR, then closes. If the bytes look like text HTTP/1.x, the server MAY answer with a plain-text HTTP/1.1 400 instead.
- If the version is not `01`, the server sends its preface and a GOAWAY VERSION_UNSUPPORTED. A client that receives a bad magic or version MUST close.

3. Frame header: fixed size, 8 bytes



- Length** is the number of payload bytes. The 8 header bytes are not counted. The range is 0 to 16,777,215.
- Type** is one of the types in §4.
- Flags**: `0x01` is END_STREAM, which marks the last frame of a message. Senders MUST set every other bit to 0, and receivers MUST ignore them.
- Stream ID**: 0 means the connection itself (GOAWAY). A request uses an id from 1 to $2^{24}-1$, and ids MUST strictly increase on a connection. Every frame of a response repeats the id of its request.

Why these widths? Eight bytes is one 64-bit word and one read(8). It holds every field without padding. There is no version field here, because the preface states the version once per connection. There is no checksum, because TCP has one, and there is no reserved bit, because HTTP/2's R bit has never been used. A 24-bit stream id leaves room for v2 to add multiplexing without changing this header. See RATIONALE.md.

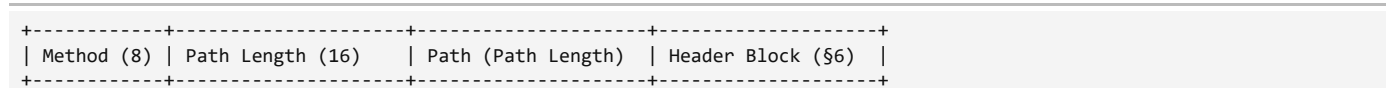
4. Frame types, and the rule that leaves room for version 2

Type	Name	Direction	Payload
0x01	REQUEST	client → server	§5
0x02	RESPONSE	server → client	§6
0x03	DATA	both	body bytes (§7)
0x04	GOAWAY	both, stream 0	§7
0x00, 0x05–0xEF	(unassigned)		reserved for future versions
0xF0–0xFF	GREASE	either	never assigned. Senders MAY send these at any time.

A receiver that meets a frame type it does not know MUST read and discard exactly Length payload bytes, then continue as if the frame had never been sent. It MUST NOT reply, MUST NOT close the connection, and MUST NOT interpret the Flags or Stream ID.

A DATA frame that arrives on a stream with no body pending is discarded in the same way. The GREASE range exists so that this skipping code actually runs in v1, and does not rot before v2 needs it.

5. REQUEST (0x01)



- Method**: `01` GET, `02` HEAD, `03` POST, `04` PUT, `05` DELETE, `06` OPTIONS. All other codes are unassigned, and the server answers them with **501**.
- Path** is UTF-8 and MUST start with `/`. It MUST NOT contain NUL. Everything after the first `?` is the query. The path is length-prefixed, so it travels **decoded**: no percent-encoding. A receiver MUST NOT percent-decode it.
- If the request has no body, the client sets END_STREAM on the REQUEST frame. If it has a body, the body follows as DATA frames on the same stream.

6. RESPONSE (0x02) and the header block

A RESPONSE payload is **Status (16)** (100–599), followed by a **Header Block**. The block fills the rest of the payload and is a sequence of fields:

+-----+ +-----+ +-----+ +-----+ +-----+				
Index (8)	Name Len (8)	Name (Index 0)	Value Len (16)	Value
+-----+ +-----+ +-----+ +-----+ +-----+				

Index 0 means a literal name follows. **Indexes 1–10** name the ten fields that BHTTP actually sends:

#	name	#	name	#	name	#	name	#	name
1	host	3	accept	5	content-length	7	date	9	etag
2	user-agent	4	content-type	6	server	8	last-modified	10	allow

- A sender **MUST** use the index for any name that is in this table, so each field has exactly one encoding. A receiver **MUST** also accept the literal form.
- A literal name is 1–255 bytes of lower-case token characters (RFC 9110 tchar). For example, x-trace: 7 is encoded as 00 07 78 2d 74 72 61 63 65 00 01 37.
- A value is 0–65,535 bytes. It **MUST NOT** contain CR, LF or NUL, so it can never inject a header line when gatewayed to HTTP/1. Field order is preserved, and duplicates are allowed.
- **Indexes 11–255 are reserved.** A receiver **MUST** skip such a field, which it can do because the value is still length-prefixed. This is §4's rule applied to header fields.
- content-length is optional. If present, it **MUST** equal the total DATA payload, and the client treats a mismatch as a protocol error.

7. DATA (0x03) and GOAWAY (0x04)

DATA: the payload is body bytes. Senders **SHOULD NOT** put more than 16,384 bytes in one DATA frame. Empty DATA frames are legal. A message ends with the first frame that has END_STREAM set. If a REQUEST or RESPONSE carries END_STREAM itself, the message has no body. HEAD, 204, 304 and OPTIONS responses work this way.

GOAWAY (stream 0): the payload is **Error Code (16)** followed by an optional UTF-8 reason. The sender closes the connection after sending it. Error codes: 0 NO_ERROR (a normal close or an idle timeout), 1 PROTOCOL_ERROR, 2 FRAME_SIZE_ERROR, 3 INTERNAL_ERROR, 4 VERSION_UNSUPPORTED. Unknown codes are treated as 1.

8. Errors: stream errors and connection errors

Stream error. The frame's Length was honoured, so the receiver still knows where the next frame starts. The server answers on that stream with a status and a text/plain reason, and **keeps the connection open**.

- **400 Bad Request** covers these cases: the payload is shorter than its fixed fields, a path length runs past the payload, the path is invalid or contains a . . segment or a backslash, a header field is truncated or illegal, the stream id is 0 or not increasing, or a RESPONSE frame was sent to the server.
- **404** means no regular file is at that path under the root. Symlinks that lead out of the root count as not found.
- **405** is for a known method other than GET, HEAD or OPTIONS. The response includes allow.
- **431** is for a REQUEST payload over 16,384 bytes. The server skips the payload first.
- **501** is for an unassigned method code.
- if-none-match that matches the etag gives **304**.

Connection error. The byte stream can no longer be trusted. This happens with a bad preface or version, an EOF in the middle of a frame, or a body the server cannot finish (v1 has no per-stream reset). The detecting side sends GOAWAY and closes. A server that is idle (30 s by default) sends GOAWAY NO_ERROR.

Client exit status: 0 OK · 2 usage error · 3 connection or protocol error · 4 any 4xx · 5 any 5xx. A client fetching several URLs from one host:port **MUST** reuse the connection.

9. Worked example: GET /index.html, stream 1, 57 bytes on the wire

00 00 31 01 01 00 00 01 01 00 0b 2f 69 6e 64 65	len=49 REQUEST END_STREAM sid=1 GET, path len 11, "/inde
78 2e 68 74 6d 6c 01 00 0e 6c 6f 63 61 6c 68 6f	x.html" [1]host len 14 "localho
73 74 3a 39 30 30 30 02 00 09 62 63 75 72 6c 2f	st:9000" [2]user-agent len 9 "bcurl/
31 2e 30 03 00 03 2a 2f 2a	1.0" [3]accept len 3 "*/"

The equivalent HTTP/1.1 text request is 86 bytes. HEXDUMP.md annotates a full captured exchange in both directions, byte by byte.